

شبکه‌های عصبی مصنوعی



مقدمه‌ای بر هوش محاسباتی و زیستی - سعید محقق

فهرست

- معرفی شبکه عصبی مصنوعی

- چرخه آموزش مدل‌ها

1. پیش پردازش داده‌ها

2. طراحی مدل

3. آموزش شبکه

- الگوریتم Back-Propagation

- توابع خطا

- الگوریتم بهینه سازی GD

- ارزیابی عملکرد مدل‌ها

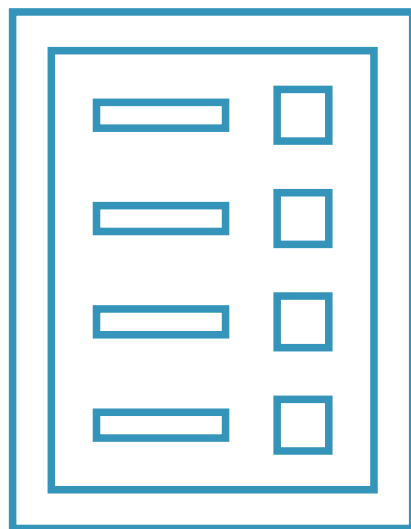
- تقسیم داده‌ها

- معیارهای ارزیابی در Regression

- معیارهای ارزیابی در Classification

- ارزیابی نمودارهای آموزش

- معرفی کتاب

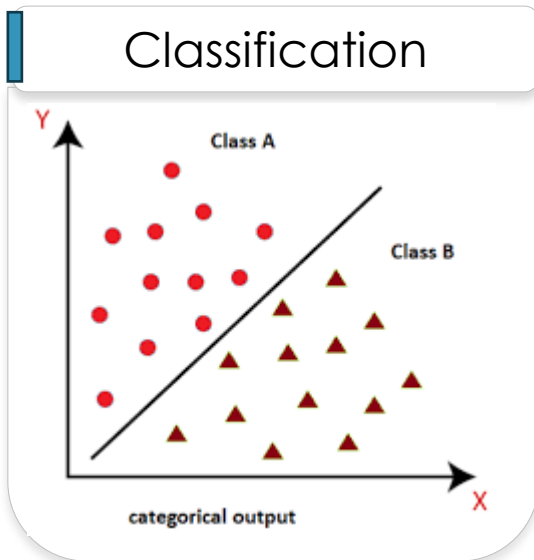


معرفی شبکه عصبی مصنوعی

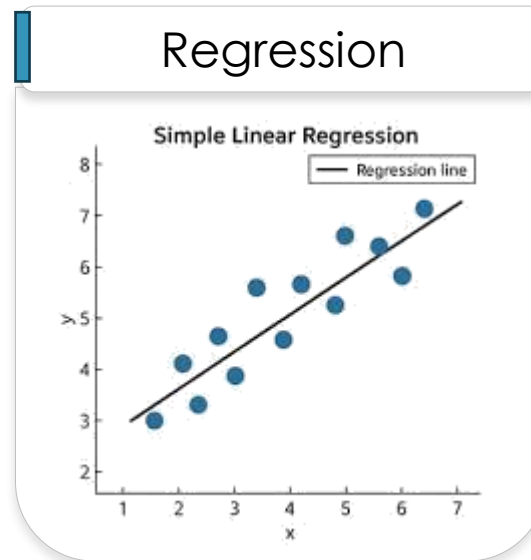
Artificial
Neural
Network

یادآوری

- روش یادگیری ← یادگیری با نظارت
- الگوریتم‌ها:
- طبقه‌بندی (Classification)
- رگرسیون (Regression)



خروجی: کلاس‌های گسسته



خروجی: مقادیر پیوسته

یادآوری

- موارد مورد نیاز:

- نمونه‌هایی از **داده ورودی** (Input Samples)

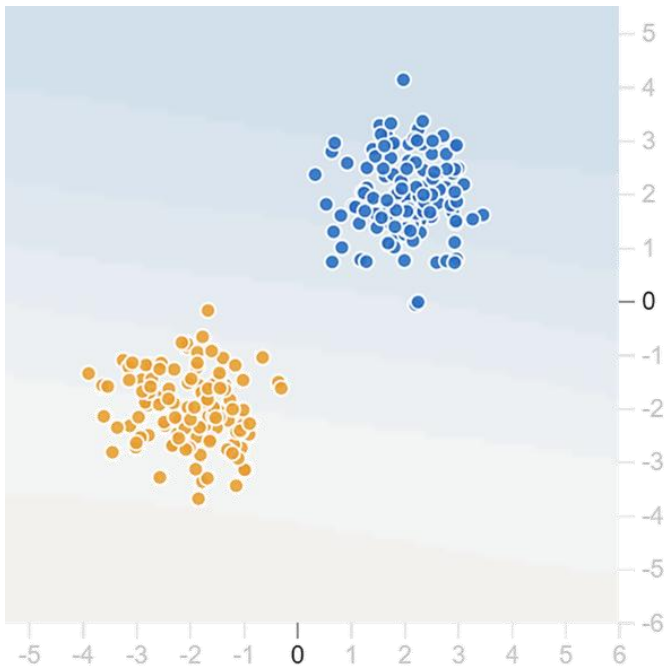
مثلاً مشخصات خانه / مشخصات نقاط

- **خروجی مطلوب** برای هر نمونه (Expected Outputs)

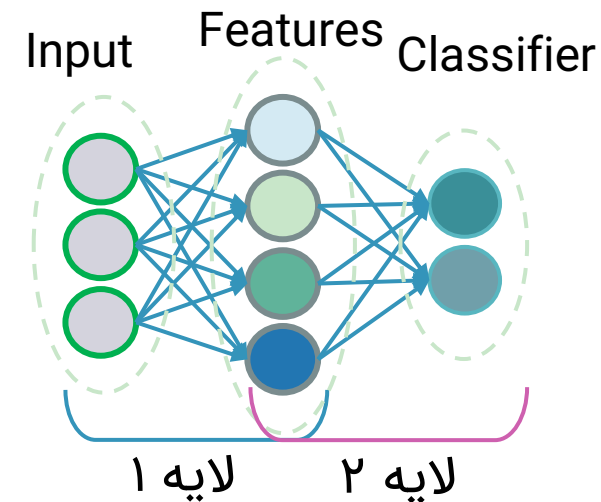
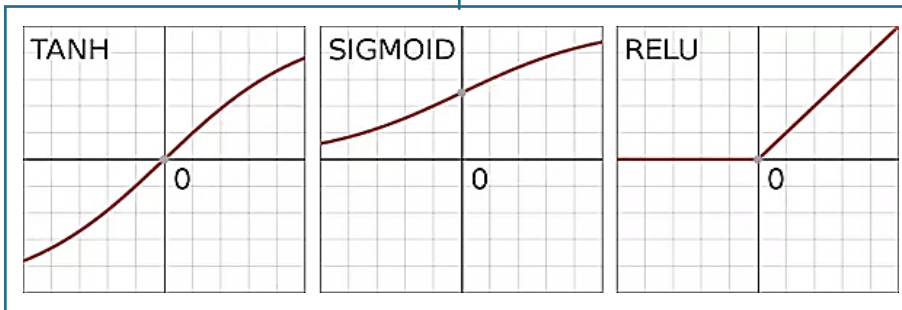
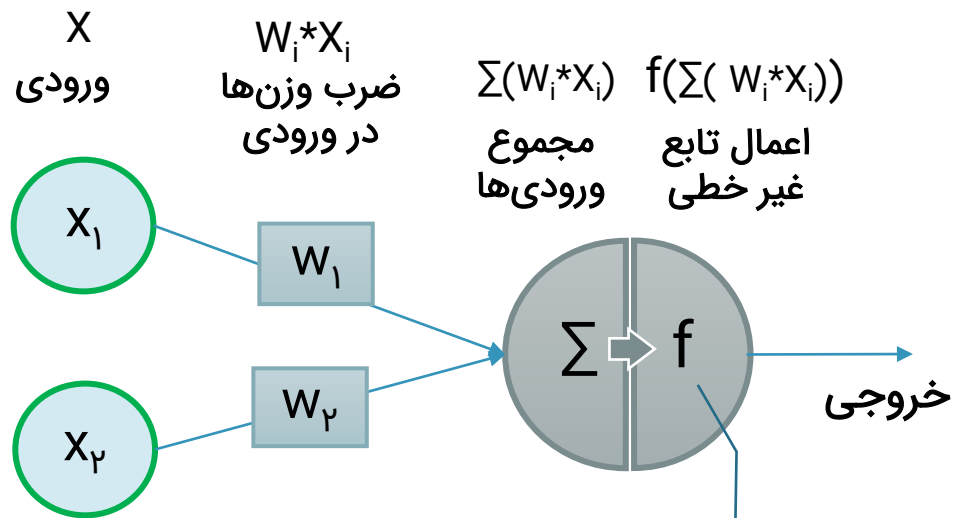
مثلاً قیمت خانه / رنگ هر نقطه

- یک **روش ارزیابی** خروجی مدل (Evaluation Metric)

مثلاً اختلاف قیمت / درصد تشخیص‌های درست



شبکه عصبی مصنوعی



شبکه عصبی
(دولایه)

تست عملی شبکه عصبی

<http://playground.tensorflow.org>

• چرخه آموزش مدل‌ها

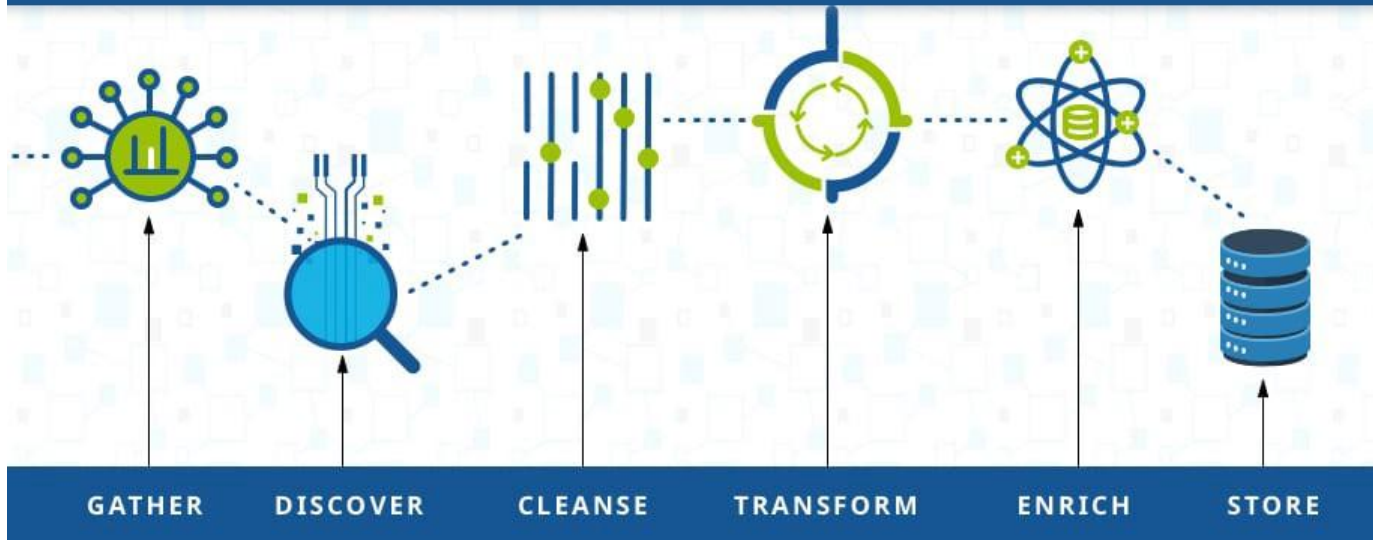
Model
Training

گام‌های اصلی فرآیند آموزش



اصول آماده‌سازی داده‌ها

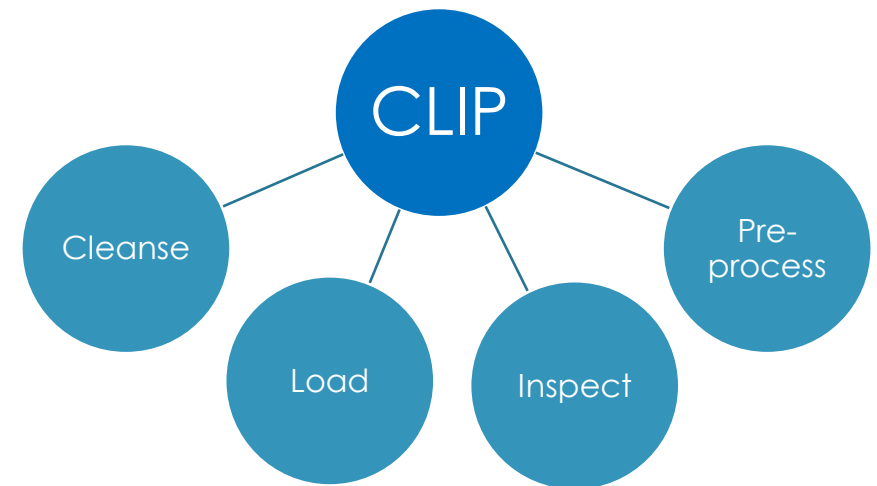
DATA PREPARATION



• داده‌ها ← منبع تغذیه و سوخت مدل

• داده خوب ← آموزش خوب

• Data-Centric AI



پیش‌پردازش داده‌ها

- نرمالیزه / استاندارد کردن داده‌ها ← حذف اطلاعات گمراه‌کننده
- Feature Scaling

$$x_{min-max} = \frac{(x - \min(x))}{(\max(x) - \min(x))}$$

`norm = (img - np.min(img)) / (np.max(img) - np.min(img))`

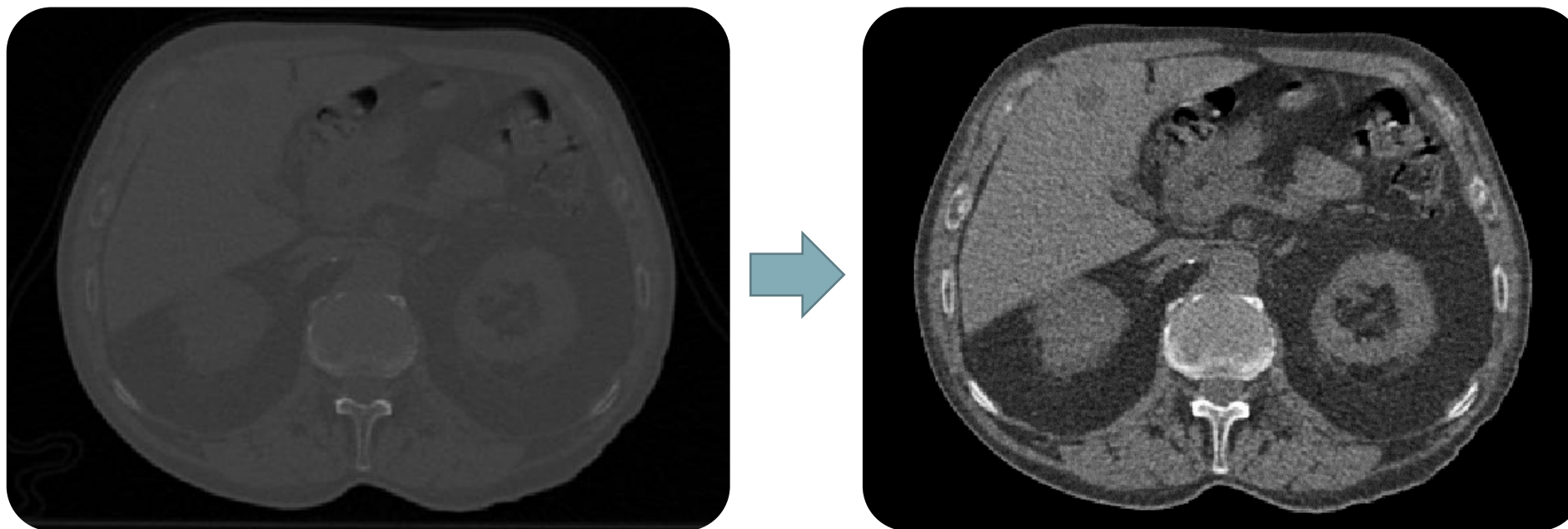
$$x_{z-score} = \frac{(x - \text{mean}(x))}{\text{std}(x)}$$

`norm = (img - np.mean(img)) / np.std(img)`

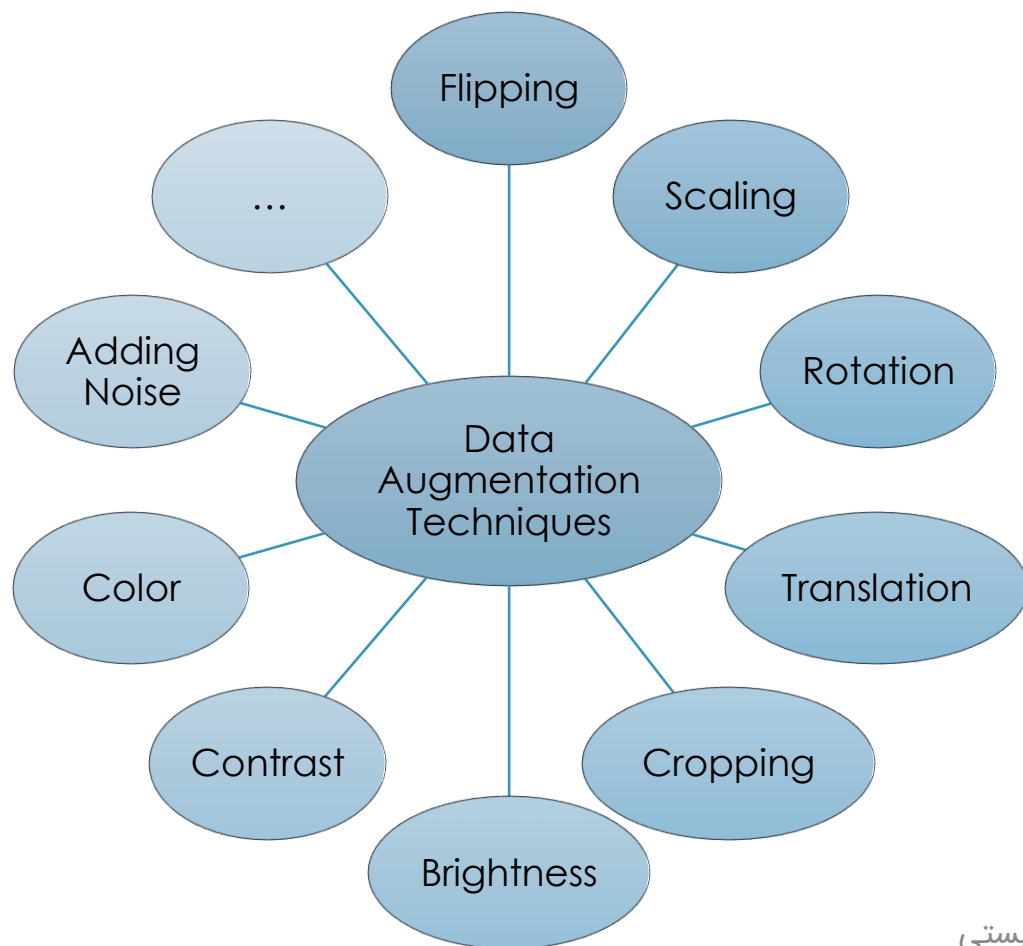
پیش پردازش داده‌ها

- بهبود کیفیت تصاویر

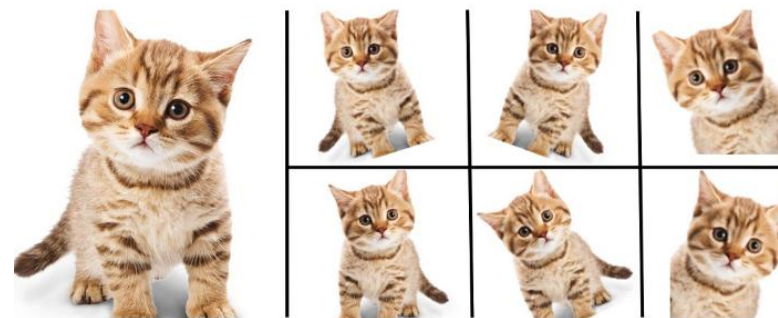
Contrast adjustment



افزایش داده‌ها



- Data Augmentation
- ایجاد تغییرات بر روی داده‌های فعلی و ایجاد داده‌های جدید



Enlarge your Dataset

انتخاب / طراحی معماری مدل

نوع و معماری شبکه

- با توجه به نوع داده های آموزش
- استفاده از معماری مدل های آماده

تعداد لایه ها / تعداد پارامترها

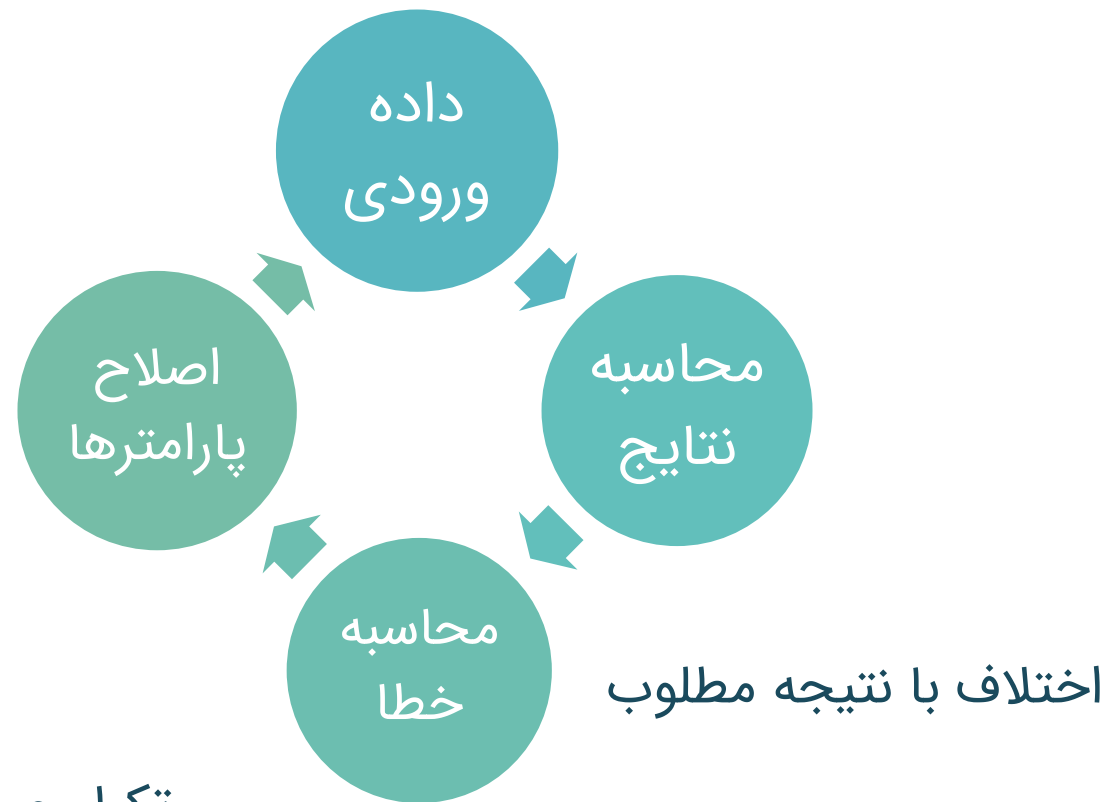
- با توجه به حجم داده های آموزش
- شروع با تعداد کم ← افزایش، تا جایی که نتیجه بهبود پیدا نکند. (یا برعکس!)

الگوریتم های جستجوی معماری (Neural Architecture Search)

- روش های AutoML (TPO, Auto-Sklearn, Auto-Keras)

آموزش شبکه

• چرخه آموزش

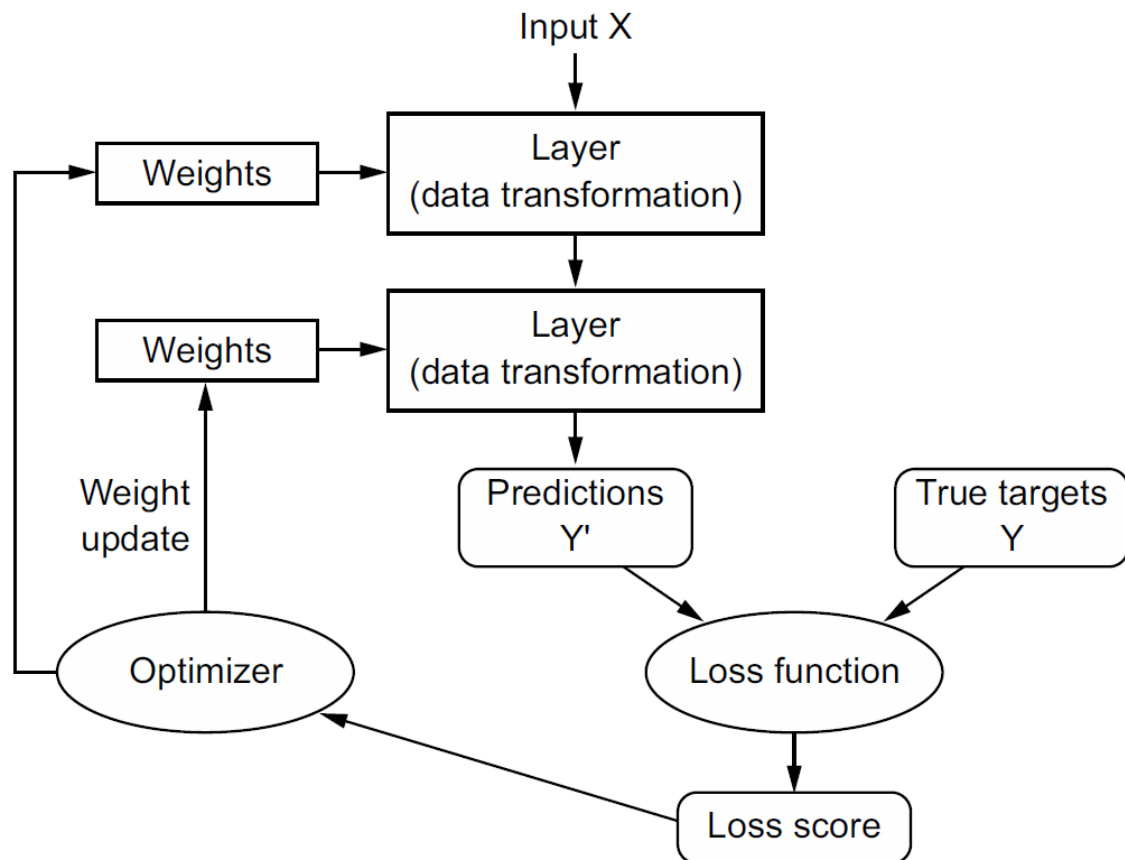


تکرار چرخه آموزش تا رسیدن به نتیجه مطلوب

الگوریتم Back-Propagation

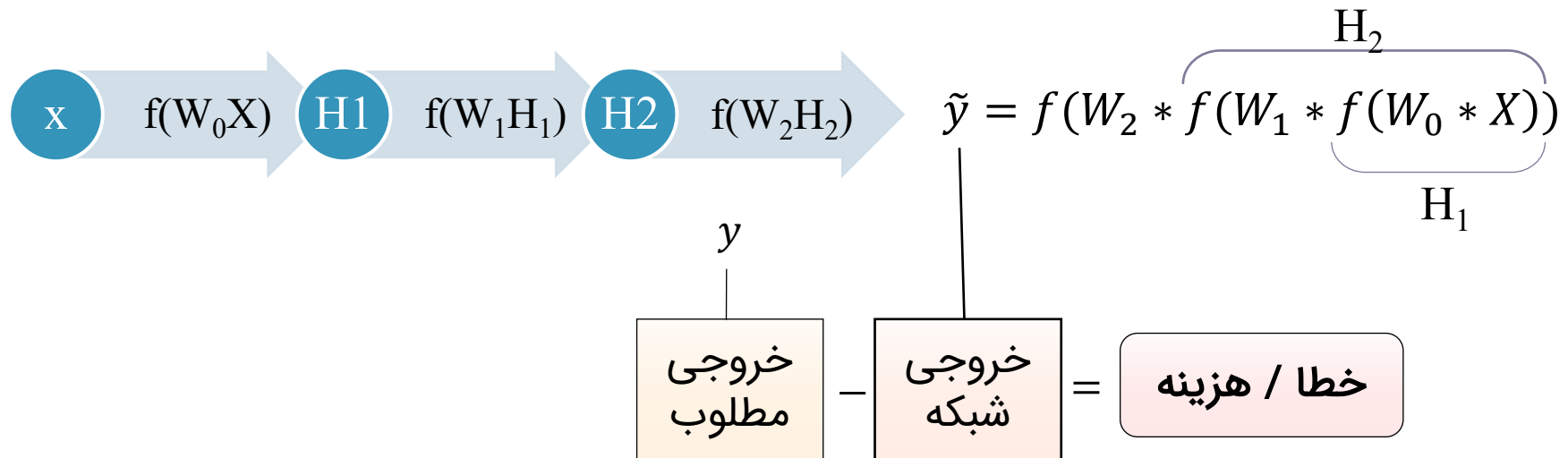
• مراحل کار:

1. محاسبه خروجی از ورودی فعلی
 2. محاسبه خطا (با تابع هزینه)
 3. اصلاح وزن‌ها توسط الگوریتم بهینه‌سازی
- تکرار مراحل با داده جدید



الگوریتم Back-Propagation

- عبور داده‌ها از شبکه و محاسبه خطا (Forward Pass)



محاسبه خطا (Loss)

- تابع هزینه ← نحوه محاسبه خطا
- نمونه‌های تابع هزینه (Cost / Loss Function)

Mean Squared Error (MSE)

$$L(y, \tilde{y}) = \frac{1}{N} \sum_i (y_i - \tilde{y}_i)^2$$

Binary Cross-Entropy (BCE)

$$L(y, \tilde{y}) = \frac{1}{N} \sum_i y_i \cdot \log(\tilde{y}_i) + (1 - y_i) \cdot \log(1 - \tilde{y}_i)$$

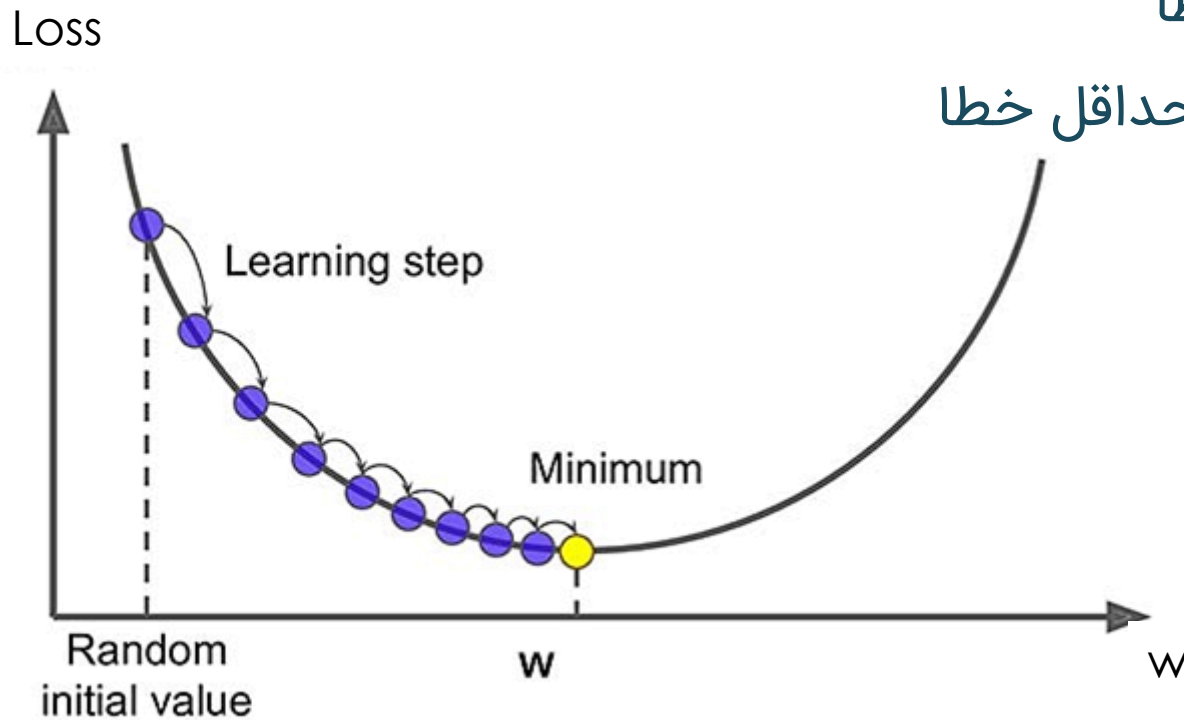
Dice Similarity Coefficient (DSC)

$$L(y, \tilde{y}) = 1 - \frac{2(y \cap \tilde{y})}{y \cup \tilde{y}}$$

بهینه‌سازی (Optimization)

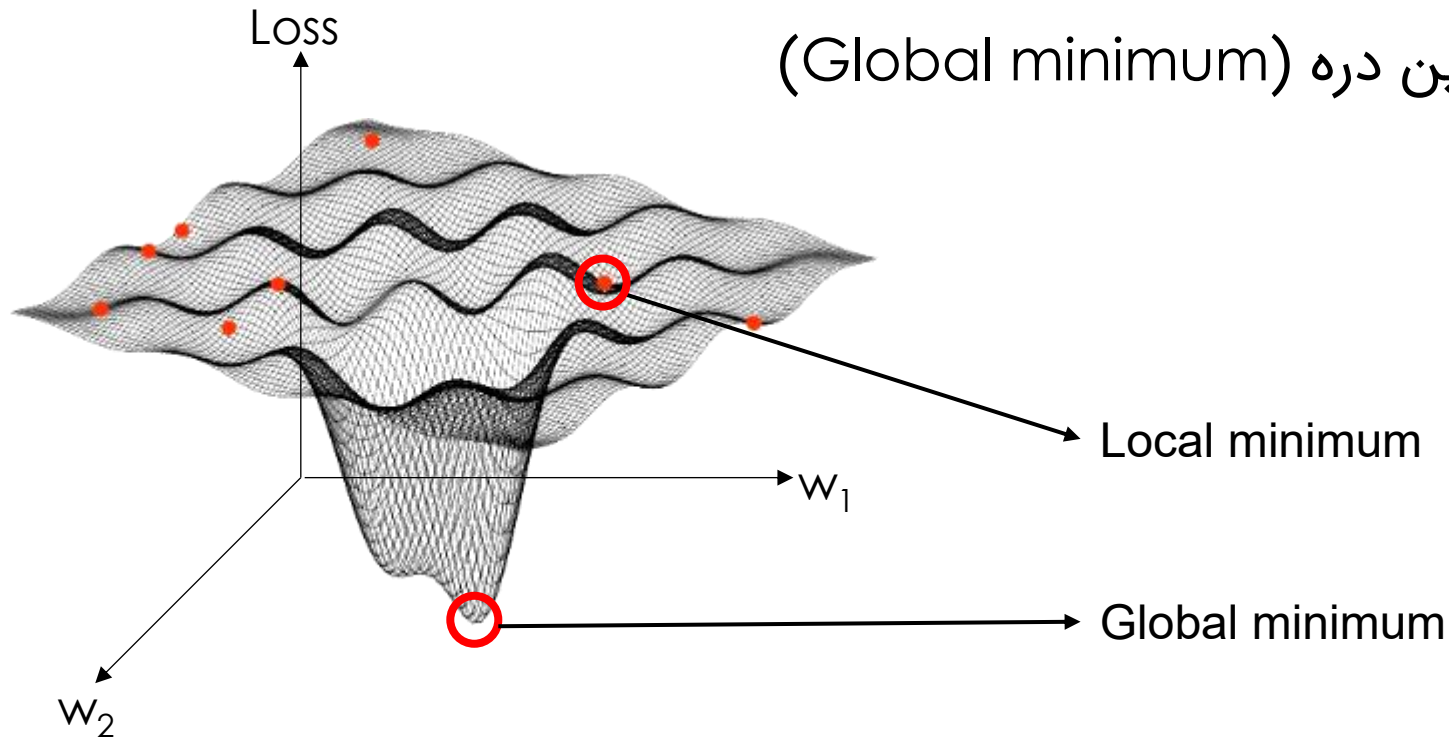
- حرکت قدم به قدم به سمت کم‌ترین مقدار خطا
 - اصلاح و به‌روزرسانی وزن‌ها برای رسیدن به حداقل خطا
- الگوریتم Gradient Descent

$$w_{i+1} = w_i - \alpha \frac{\partial Loss}{\partial w_i}$$



بهینه‌سازی (Optimization)

- مقادیر مختلف Loss در وزن‌های مختلف $\leftarrow$ تشکیل یک سطح ناهموار (Loss surface)
- هدف $\leftarrow$ رسیدن به عمیق‌ترین دره (Global minimum)



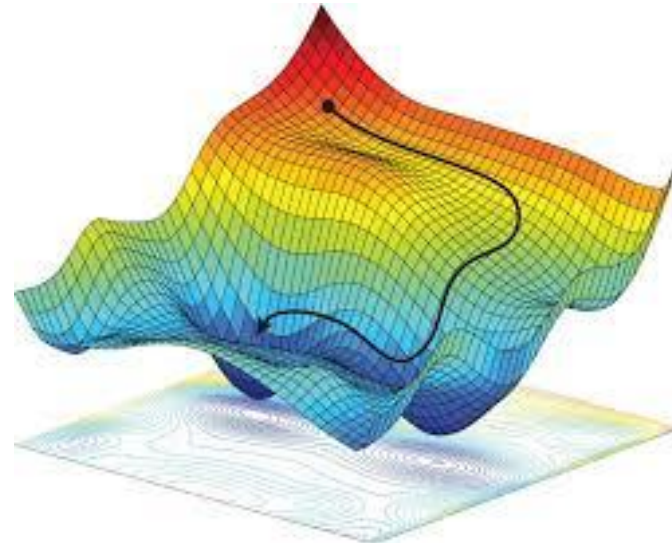
بهینه‌سازی (Optimization)

الگوریتم‌ها

- SGD
- SGD + Momentum
- RMSprop
- Adagrad
- Adadelta
- Adam
- ...

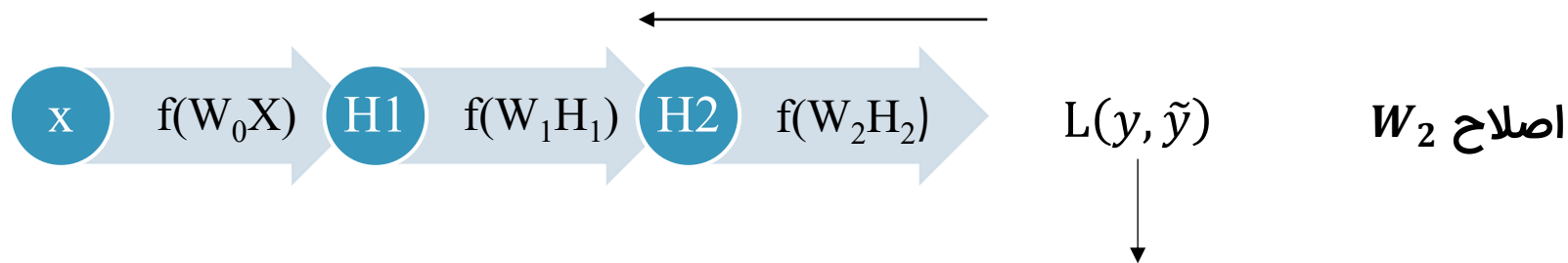
• بررسی الگوریتم‌های بهینه‌سازی:

<https://emiliendupont.github.io/2018/01/24/optimization-visualization/>



اصلاح وزن‌ها

• روش Back-propagation (Backward Pass)



$$\Delta W_2 = \frac{\partial L}{\partial W_2}$$

$$W_2' = W_2 - \alpha(\Delta W_2)$$

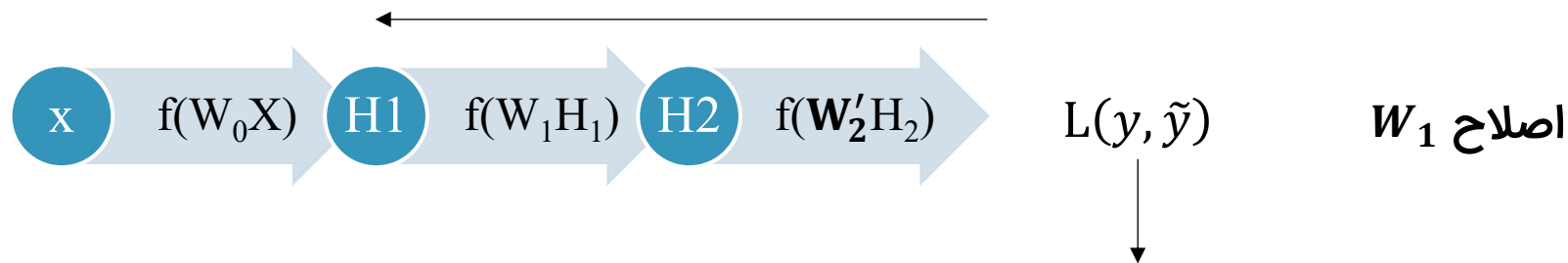
↓
Learning rate

• محاسبه گرادیان L نسبت به وزن W_2 :

• اصلاح W_2 در جهت کاهش گرادیان L (الگوریتم SGD):

اصلاح وزن‌ها

• روش Back-propagation (Backward Pass)



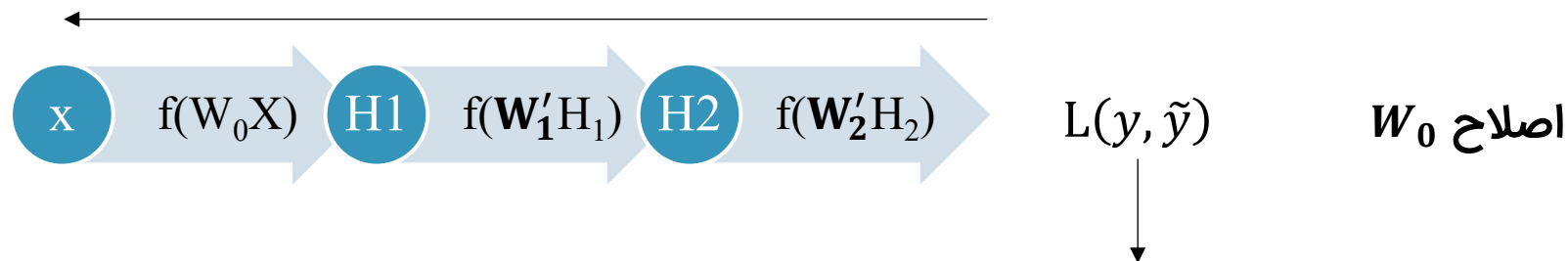
$$\Delta W_1 = \frac{\partial L}{\partial W_1}$$

$$W_1' = W_1 - \alpha(\Delta W_1)$$

- محاسبه گرادیان L نسبت به وزن W_1 :
- اصلاح W_1 در جهت کاهش گرادیان L :

اصلاح وزن‌ها

• روش Back-propagation (Backward Pass)



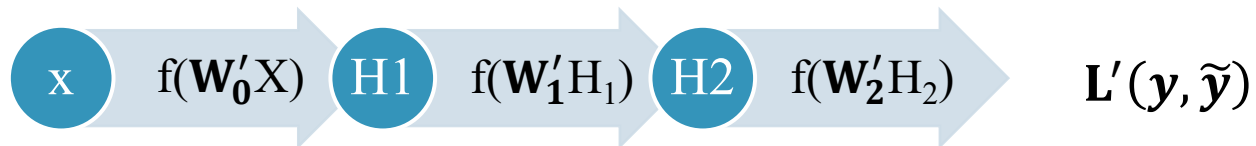
$$\Delta W_0 = \frac{\partial L}{\partial W_0}$$

$$W'_0 = W_0 - \alpha(\Delta W_0)$$

- محاسبه گرادیان L نسبت به وزن W_0 :
- اصلاح W_0 در جهت کاهش گرادیان L :

اصلاح وزن‌ها

• روش Back-propagation



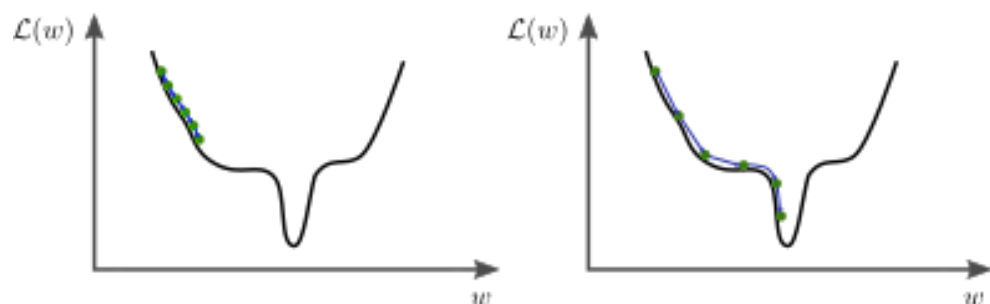
→ محاسبه مجدد خطا (Forward Pass)

← اصلاح وزن‌ها با خطای جدید (Backward Pass)

- هر مرحله از مسیر رفت و برگشت ← یک Iteration (شامل یک گروه یا Batch از داده ها)
- هر مرحله عبور تمام داده‌ها از مدل ← یک Epoch (شامل یک یا چند Iteration)

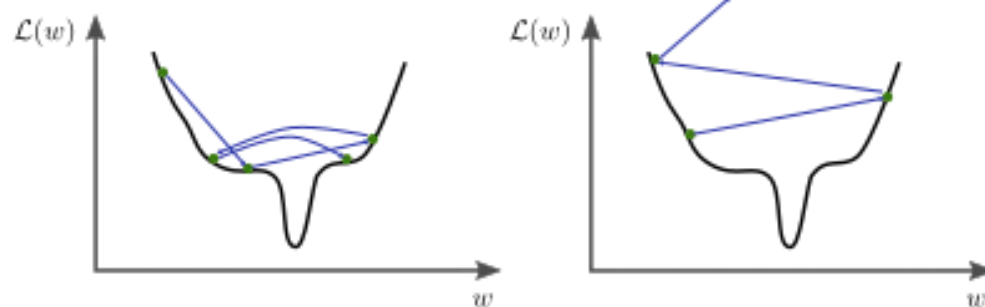
نرخ یادگیری

Learning Rate •



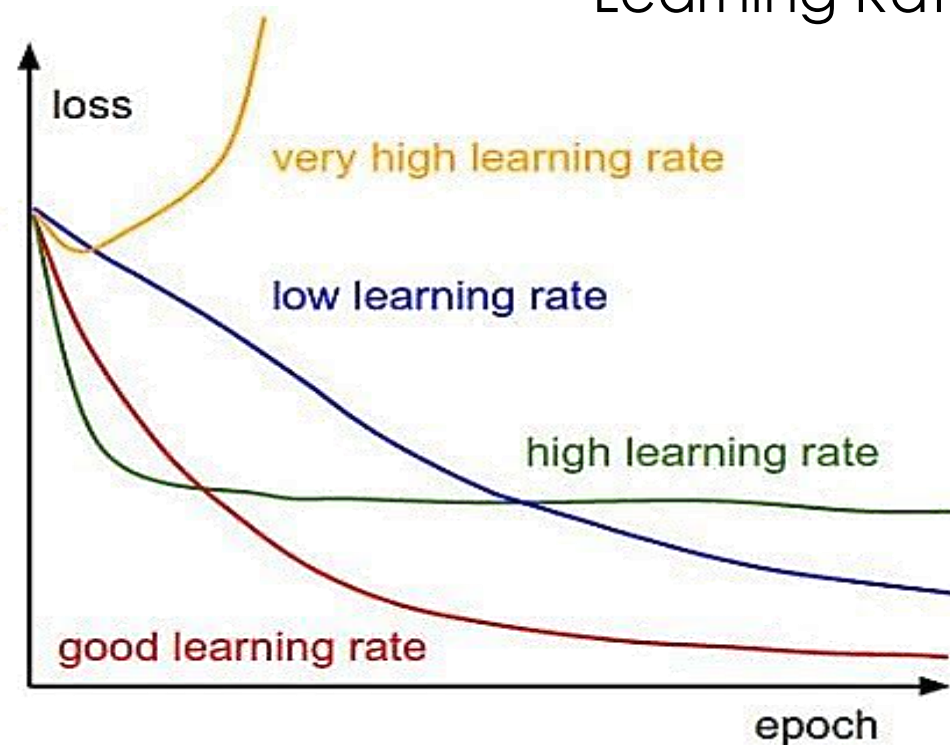
Too low

Good



High

Too high

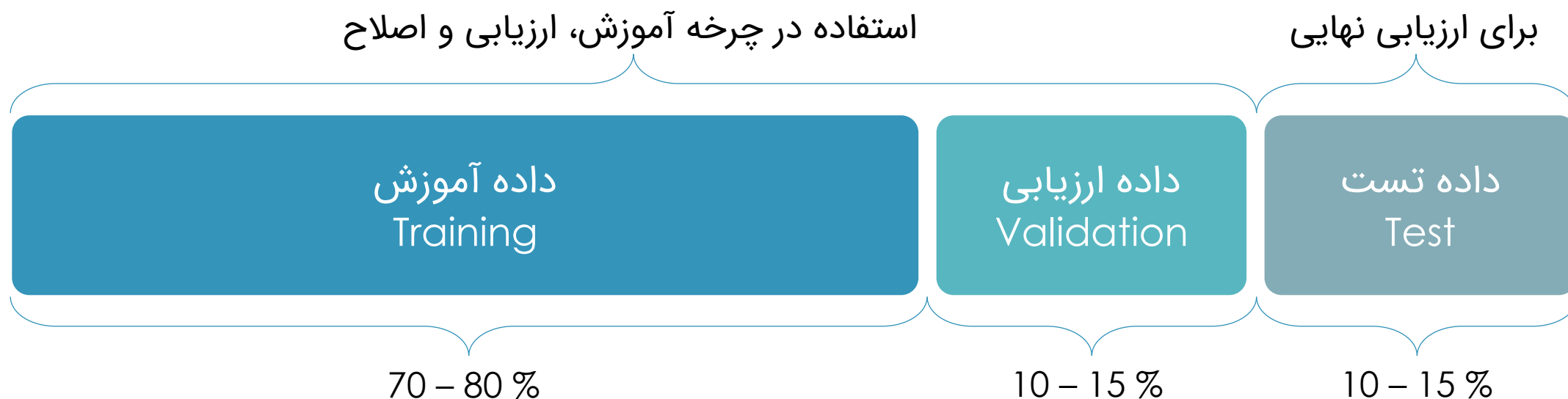


• ارزیابی مدل‌ها

Model
Evaluation

تقسیم داده‌ها

- داده‌های موجود را به سه گروه، تقسیم می‌کنیم:



- معیارهای ارزیابی، برای داده‌های Validation و Test محاسبه می‌شوند.

معیارهای ارزیابی عددی

Classification

- Accuracy
- Confusion Matrix
- Precision
- Recall
- F1-Score

Regression

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- Mean Absolute Percentage Error (MAPE)
- R-Squared (R^2)

محاسبه معیارهای ارزیابی

- خروجی مدل برای داده‌های validation بعد از هر epoch، و برای داده‌های Test در انتهای آموزش، محاسبه می‌شود.
- معیارها بین خروجی مدل (y_{pred}) با برچسب‌ها یا خروجی صحیح (y_{true})، محاسبه می‌شود.
- معیارهای عددی معمولاً به صورت تابعی از دو متغیر، تعریف می‌شوند: $F(y_{true}, y_{pred})$

• $y_{pred} \rightarrow$ لیستی از خروجی‌های مدل

0.1	0.9	0.8	0.0	0.2	0.5	0.7
-----	-----	-----	-----	-----	-----	-----

• $y_{true} \rightarrow$ لیستی از مقادیر برچسب‌ها

0	1	1	0	0	0	1
---	---	---	---	---	---	---

نمونه کد محاسبه معیارها (با استفاده از Scikit-Learn)

```
from sklearn.metrics import accuracy_score, f1_score, mean_squared_error, r2_score
```

Classification

```
y_pred = model.predict(X_test)

acc = accuracy_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred)
...
```

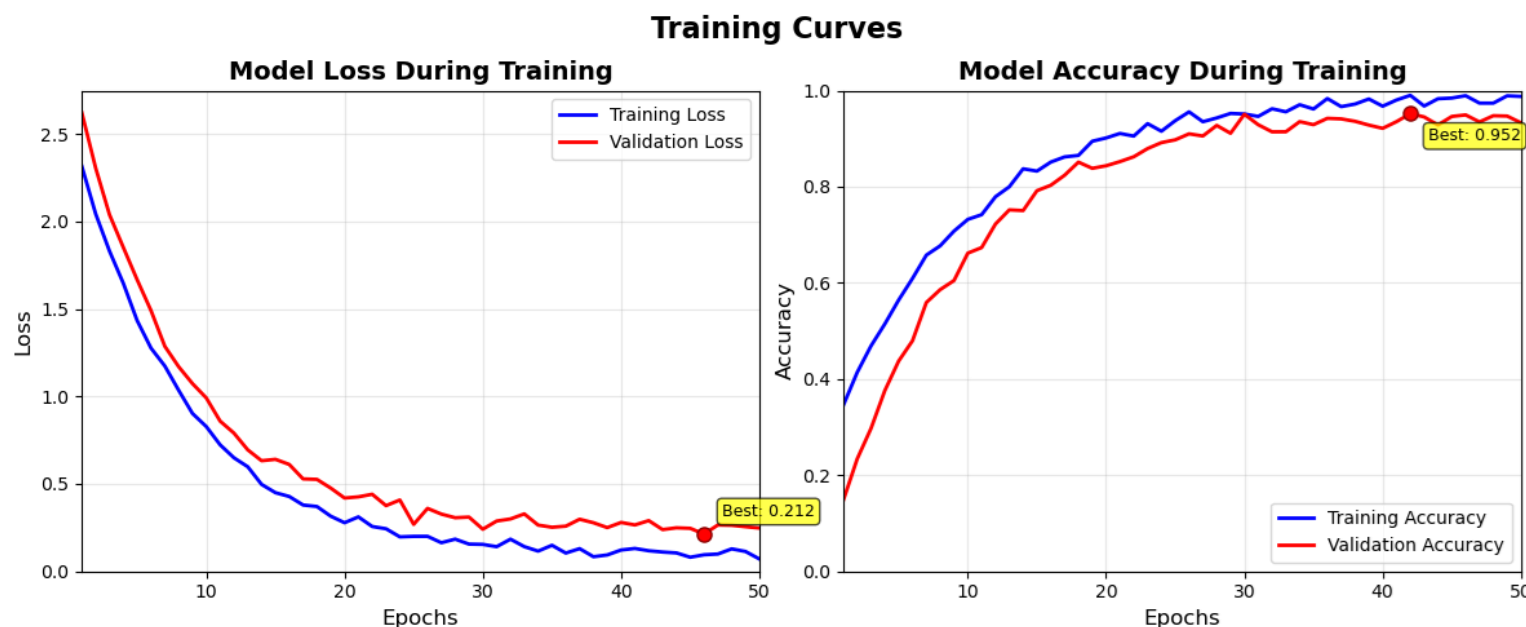
Regression

```
y_pred = model.predict(X_test)

mse = mean_squared_error(y_true, y_pred)
r2 = r2_score(y_true, y_pred)
...
```

ارزیابی نمودارهای آموزش

- تجزیه و تحلیل نمودار مقادیر خطا (Loss) و معیارهای دیگر در طول آموزش
- مشاهده رابطه بین نمودارهای داده‌های Train و Validation برای تشخیص Over-fitting



مزایا و چالش‌های شبکه‌های عصبی

چالش‌ها

- پشتوانه تئوری ضعیف
- هزینه محاسباتی بالا
- نیاز به تعداد زیاد داده
- دشواری تنظیم پارامترها
- مشکلات آموزش



مزایا

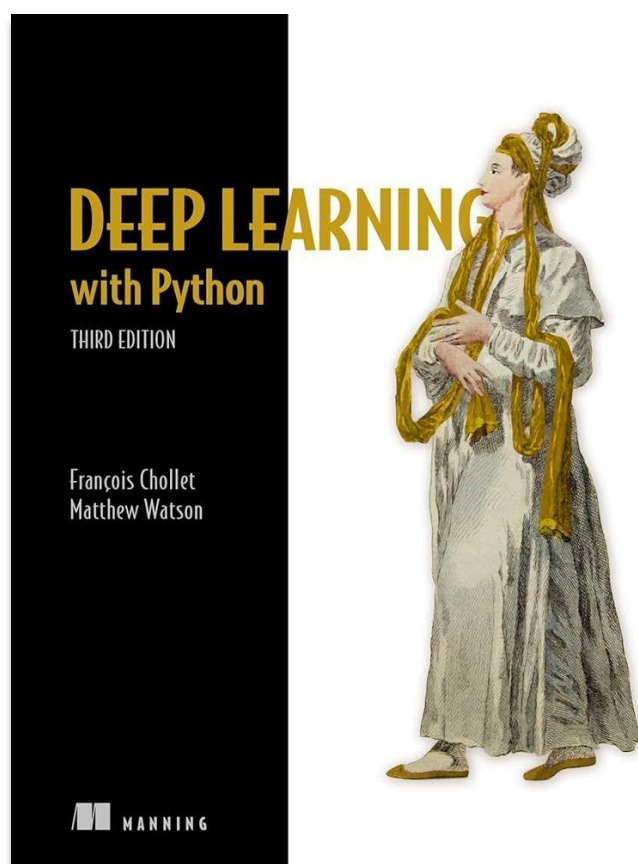
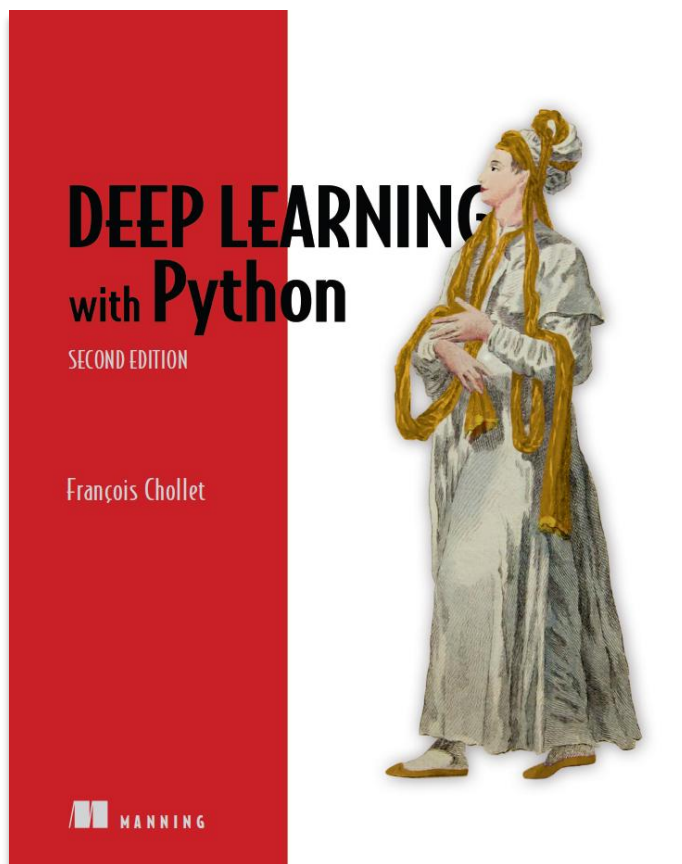
- یادگیری خودکار ویژگی‌ها
- یادگیری چندلایه ویژگی‌ها
- دقت بالا
- پشتیبانی سخت‌افزاری و نرم‌افزاری
- پتانسیل ایجاد قابلیت‌های بیشتر

منبع کمکی

• کتاب

- Deep Learning with Python
 - François Chollet

• ویرایش دوم (۲۰۲۱) یا سوم (۲۰۲۵)



با تشکر / Thank you



سعید محققى / Saeed Mohagheghi



Daneshjoy.ir@gmail.com

s.mohagheghi@shahed.ac.ir



[@SAEED_MHQ](#) (Rubika / Bale / Telegram)

<https://github.com/daneshjoy>

• ویدئوهای این درس در آپارات



<https://aparat.com/v/sfpw7r6>



<https://aparat.com/v/cyqx092>



<https://aparat.com/v/wsu76p3>